



UNIVERSITY OF MINES AND TECHNOLOGY
FACULTY OF COMPUTING AND MATHEMATICAL SCIENCES

CY376: NETWORK MONITORING, SECURITY AND AUDITING
END-OF-SEMESTER PROJECT REPORT

Building a File Integrity Monitoring System for Critical Servers

A Blue Team Defensive Security Project

Prepared by: Akyempim-Boamah Abena Owusua

Index Number: FCM.41.018.035.23

Topic Title: Building a File Integrity Monitoring System for Critical Servers

Team: Blue Team

Course Code: CY376

Date: 3rd August 2026

GitHub Repository: [Insert your GitHub repository link here]

Abstract

File Integrity Monitoring (FIM) is a foundational blue-team control that detects unauthorized changes to critical system files, such as configuration files, credential stores, system binaries, and scheduled tasks. Attackers who gain a foothold on a server frequently need to modify one of these files, for example to add a backdoor account, install a malicious cron job, replace a system binary, or weaken SSH configuration, so a control that reliably surfaces these changes closes off a step that many intrusions depend on.

This project designed, built, and tested a working FIM system from first principles rather than deploying an existing tool such as AIDE or Tripwire directly, in order to demonstrate a practical understanding of the detection logic, tamper-resistance requirements, and operational deployment considerations involved. The resulting tool, `fim.py`, records a cryptographic baseline of a defined set of Linux configuration files and metadata, protects that baseline against silent tampering with a self-hash, and supports both scheduled polling and real-time, event-driven detection.

The system was tested in an isolated virtual machine against five scenarios: a clean-state check, a simulated backdoor-account attack, a baseline tamper-resistance test, unattended operation via a systemd timer, and real-time detection using the inotify-based watchdog library. All five produced the expected results, and one operational limitation of state-based comparison, namely that changes reverted between checks leave no trace in a single scan, was identified and documented along with mitigations. The report closes with an analysis of what the results mean for real deployments and a set of recommendations for hardening the system further, including offloading the baseline to a separate host and forwarding alerts to a centralized SIEM.

Table of Contents

Abstract.....	2
1. Introduction / Objective	4
2. Literature and Tooling Review.....	4
2.1 Adversary behaviour: MITRE ATT&CK.....	5
2.2 Configuration and compliance baselines.....	5
2.3 Existing FIM tools.....	5
3. Methodology	6
3.1 Test environment.....	6
3.2 Directory layout.....	6
3.3 Configuration.....	7
3.4 System design.....	7
4. Implementation	8
4.1 Baseline generation	8
4.2 Tamper-evidence (do_verify).....	8
4.3 Change detection (do_check)	8
4.4 Real-time monitoring (do_watch)	9
4.5 Alerting.....	9
4.6 Automation.....	9
5. Results and Findings.....	10
5.1 Baseline creation	10
5.2 Clean-state baseline check.....	11
5.3 Simulated attack: unauthorized account creation	12
5.4 Finding: reverted changes are not retroactively visible.....	12
5.5 Baseline tamper-resistance test.....	12
5.6 Real-time detection (watch mode).....	13
5.7 Automated, unattended checks	13
5.8 Summary of test results	14
6. Analysis and Recommendations	14
6.1 What the results mean	14
6.2 Recommendations	15
7. Conclusion	16
References.....	17
Appendix A: Full Test Timeline (Log Evidence)	18
Appendix B: Full Source Code Reference	19

1. Introduction / Objective

File Integrity Monitoring (FIM) is a core blue-team control that detects unauthorized changes to critical system files: configuration files, credential stores, system binaries, and scheduled tasks. An attacker who gains a foothold on a server frequently needs to modify one of these files, for example by adding a backdoor user, installing a malicious cron job, replacing a system binary, or weakening SSH configuration. FIM provides an early-warning capability by recording a cryptographic fingerprint of each protected file and alerting whenever that fingerprint changes unexpectedly, closing off a step that a large share of post-exploitation activity depends on.

Beyond detecting a single suspicious edit, FIM plays a broader defensive role. It gives an incident responder a timestamped record of exactly which files changed, when, which is often the difference between confirming a compromise quickly and spending hours manually diffing configuration against backups. It also acts as a deterrent and a compliance control in its own right: several widely used security baselines, discussed in Section 2, either recommend or require periodic integrity checking of critical files as a matter of course.

This project's objective was to design, build, and test a working FIM system from first principles rather than relying solely on an off-the-shelf tool, in order to demonstrate a practical understanding of the detection logic, tamper-resistance requirements, and operational deployment considerations involved. Building the tool in-house, rather than simply installing AIDE or Tripwire and pointing it at a directory, forced explicit decisions about what to hash, how to protect the baseline itself from tampering, how to balance detection latency against system load, and how to get alerts out of the host reliably. Those decisions, and the reasoning behind them, are the core contribution documented in this report.

Scope

- Monitor a defined set of critical Linux configuration files: `/etc/passwd`, `/etc/shadow`, `/etc/ssh/sshd_config`, and `/etc/hosts`.
- Detect file additions, deletions, content modifications, and permission or ownership changes.
- Protect the integrity baseline itself from tampering.
- Support both scheduled (polling) and real-time (event-driven) detection.
- Provide local and external alerting channels.
- Automate checks so the system runs unattended.

The remainder of this report is organized as follows. Section 2 reviews the standards and existing tools that shaped the design. Section 3 describes the methodology, including the lab environment and the steps taken to build and configure the system. Section 4 covers the implementation of each operating mode. Section 5 presents the results of five test scenarios. Section 6 analyzes what those results mean and what should change as a result. Section 7 concludes, and full log evidence and source code are provided in the appendices.

2. Literature and Tooling Review

The design of this system was informed by three bodies of work: adversary behaviour frameworks that describe why file integrity matters, configuration and compliance baselines that formalize it as a control, and existing open-source FIM tools whose design trade-offs this project deliberately learned from and, in places, diverged from.

2.1 Adversary behaviour: MITRE ATT&CK

The MITRE ATT&CK framework catalogues adversary techniques observed in real intrusions, and several of them map directly onto the file changes this project set out to detect. Technique T1565, Data Manipulation, describes how adversaries insert, delete, or manipulate data at rest or in transit to influence outcomes or hide activity, thereby threatening the integrity of that data [1]; its Stored Data Manipulation sub-technique is the closest match to an attacker directly editing a file such as `/etc/passwd` or `/etc/shadow` to plant a backdoor account. ATT&CK's own guidance on defending against this category of technique points toward exactly this project's approach: inspecting important file hashes, locations, and modifications for suspicious or unexpected values, ideally through automated tooling rather than manual review [1].

Two related techniques shaped which files were chosen for monitoring. T1543, Create or Modify System Process, covers adversaries creating or modifying system-level processes, such as services or scheduled jobs, to run malicious code with elevated privileges, which motivates watching startup and scheduling paths in addition to account files [2]. T1036, Masquerading, covers attackers disguising malicious artefacts as legitimate ones, for example by tampering with a configuration file to look unchanged while quietly weakening it, such as loosening `sshd_config` [3]. Both reinforce that FIM should not be limited to a single file but to a small, deliberately chosen set of high-value paths where post-exploitation activity concentrates.

2.2 Configuration and compliance baselines

Beyond adversary tradecraft, integrity checking appears directly in mainstream hardening and compliance guidance. The CIS Benchmarks, published by the Center for Internet Security as consensus-built secure configuration baselines, include a specific recommendation in the CIS Distribution Independent Linux benchmark to ensure filesystem integrity is regularly checked, on the basis that periodic checking is needed to let an administrator determine on a regular basis whether critical files have changed [4]. This project's scheduled polling mode, described in Sections 3 and 4, was designed to satisfy that recommendation directly.

At a higher level, NIST SP 800-53 codifies the same requirement as control SI-7, Software, Firmware, and Information Integrity, which calls for organizations to employ integrity verification tools to detect unauthorized changes to defined software, firmware, and information, and to take a defined response action when such changes are found [5]. The control's first enhancement, SI-7(1), goes further and requires that the integrity check itself run at startup, at defined transitional states or security-relevant events, or at a defined frequency [5], which is precisely the distinction this project drew between the scheduled and real-time detection modes described in Section 4.

2.3 Existing FIM tools

Two long-standing open-source tools define the space this project's tool sits in. AIDE (Advanced Intrusion Detection Environment) builds a database of file attributes and hashes and compares against it on demand or on a schedule; it is lightweight, widely packaged on Linux distributions, and is the tool most CIS Benchmark remediation steps for filesystem integrity checking point administrators toward [6]. Tripwire, in both its open-source and commercial forms, takes a similar snapshot-and-compare approach but adds centralized policy management and reporting aimed at larger, multi-host estates. OSSEC and its actively maintained fork Wazuh extend the same idea into a broader host-based intrusion detection platform, bundling FIM together with log analysis, rootkit detection, and active response, with alerts typically forwarded to a central manager for correlation across many hosts [7].

All three share the same core weakness this project treats as a first-class design requirement rather than an afterthought: a state-based tool is only as trustworthy as its baseline, and a baseline stored unprotected on the same host it monitors can, in principle, be rewritten by a sufficiently privileged attacker to hide their own tampering. AIDE and Tripwire both address this operationally, by recommending the baseline database be signed or stored off-host, but leave it as a deployment choice rather than a property enforced by the tool itself by default. Building a custom tool made it possible to make baseline tamper-evidence a hard requirement of every check rather than an optional hardening step, as described in Section 3.4 and validated in Section 5.5.

For real-time detection, this project used Python's watchdog library, which wraps the Linux kernel's inotify subsystem to deliver filesystem events (create, modify, delete, move) with minimal latency, avoiding the detection delay inherent to any polling-based approach [8]. Combining an inotify-based watch mode for the highest-value paths with periodic full-scan polling for broader coverage reflects the same two-tier pattern used by commercial EDR and FIM products, where real-time hooks cover the paths that matter most and scheduled scans provide a safety net for everything else.

3. Methodology

Testing was performed in an isolated virtual machine rather than a production server, to safely simulate attacker behaviour, such as modifying `/etc/passwd`, without risk to a real system. The methodology below covers the lab environment, the directory and configuration layout used, and the system design decisions that shaped the implementation described in Section 4.

3.1 Test environment

The tool was developed and tested on a single Ubuntu Linux virtual machine, isolated from any production network, with root access available for installing the tool under a locked-down directory and for simulating privileged attacker actions such as `useradd` and `userdel`.

Figure 2: Test environment OS version

3.2 Directory layout

The tool and its data were installed under a locked-down directory structure, separating the executable and configuration from the baseline data and the log output, and restricting each to the minimum permissions it needed (Table 1).

/opt/fim/	- fim.py, fim_config.json	(chmod 700)
/var/lib/fim/	- baseline.json, baseline.json.sha256	(chmod 700)
/var/log/fim/	- fim.log	(chmod 750)

Table 1: Directory and file layout for the FIM installation

```

drwx----- 2 root root 4096 Aug 2 18:34 .
drwxr-xr-x 3 root root 4096 Aug 2 18:34 ..
rw-r--r-- 1 root root 328 Aug 2 18:34 fim_config.json
rw-r--r-- 1 root root 12699 Aug 2 18:34 fim.py
bena-owusua@bena-owusua-VMware-Virtual-Platform:~/fim-build$

```

Figure 3: /opt/fim/ directory listing showing fim.py and fim_config.json in place with root-only permissions

3.3 Configuration

The monitored paths and alert settings were defined in a single JSON configuration file, fim_config.json (Table 2), so that the set of protected files could be changed without editing the tool's source code.

```

{
  "monitor_paths": [
    "/etc/passwd",
    "/etc/shadow",
    "/etc/ssh/sshd_config",
    "/etc/hosts"
  ],
  "exclude": ["/proc", "/sys", "/tmp", ".log"],
  "baseline_file": "/var/lib/fim/baseline.json",
  "baseline_hash_file": "/var/lib/fim/baseline.json.sha256",
  "log_file": "/var/log/fim/fim.log"
}

```

Table 2: fim_config.json monitoring configuration

A minimal, high-value initial monitoring set was chosen so the detection logic could be validated quickly, in line with the ATT&CK-derived priorities discussed in Section 2.1, before scaling up to full system directories such as /bin, /sbin, and /usr/bin.

3.4 System design

The system is built around three core operating modes and a tamper-evidence mechanism that protects the integrity baseline itself, since a FIM tool whose baseline can be silently rewritten by an attacker provides no real security guarantee, as discussed in Section 2.3.

Figure 1: High-level architecture

The core design decisions were as follows, and are referenced again in Section 4 where each is implemented and in Section 5 where each is tested:

- SHA-256 used for content hashing: collision-resistant and fast enough for periodic full scans.
- Metadata tracked per file: size, permission mode, owner UID/GID, mtime, and symlink target.
- Baseline tamper-evidence: a separate hash of the entire baseline file is stored alongside it (see Table 2). Every check first re-hashes the baseline and compares it to the stored value; if they don't match, the tool refuses to trust the baseline and raises an alert instead of silently proceeding.
- Two detection modes: scheduled polling (a systemd timer, every 5-15 minutes) for broad coverage, and real-time event-driven monitoring (inotify via the Python 'watchdog' library) for the highest-value targets where even a short detection delay is unacceptable.
- Alerting fans out to a local log file, syslog (for SIEM forwarding), and optionally a webhook or email.

4. Implementation

The tool (fim.py) was implemented in Python 3 using only the standard library plus the optional 'watchdog' package for real-time monitoring. It exposes four modes: baseline, check, verify, and watch, plus the systemd automation described in Section 4.6.

4.1 Baseline generation

do_baseline() walks every configured path, computes a SHA-256 hash and metadata snapshot for every file found, and writes the result to baseline.json. A hash of that baseline file is then computed and stored separately in baseline.json.sha256, so that any later change to the baseline itself can be detected before it is trusted (Section 4.2).

4.2 Tamper-evidence (do_verify)

Before any comparison is trusted, do_verify() re-hashes baseline.json and compares it against the stored hash from baseline.json.sha256. Any mismatch, meaning the baseline itself was altered outside of the tool, immediately raises an ALERT and halts the check rather than comparing against a baseline that may already have been rewritten to hide an intrusion. This directly implements the tamper-evidence design decision from Section 3.4 and is validated in Section 5.5.

4.3 Change detection (do_check)

do_check() re-walks the monitored paths, computes fresh metadata and hashes, and classifies every file as unchanged, added, deleted, modified (hash differs), or permission/ownership

changed, relative to the baseline. This is the mode invoked both manually and by the systemd timer described in Section 4.6.

4.4 Real-time monitoring (do_watch)

do_watch() uses the 'watchdog' library to subscribe to filesystem events (create, modify, delete, move) on the monitored paths, evaluating each event against the baseline the moment it fires, rather than waiting for the next scheduled check. This mode is what closes the polling-delay gap identified in Section 2.3 and demonstrated in Section 5.4.

4.5 Alerting

log_event() writes every event to the local log file and syslog. For ALERT-level events, it additionally attempts a webhook POST and/or SMTP email if configured, with failures in either channel logged but never blocking the core integrity check, so that a misconfigured notification channel cannot cause a real change to go unrecorded.

The script was authored directly on the target virtual machine, under /opt/fim/, before its configuration file was created alongside it and its permissions were locked down as shown in Section 3.2 (Figure 3).



Figure 4: fim.py being created under /opt/fim/ in the nano editor

4.6 Automation

To operate unattended, the check mode was scheduled using a systemd service/timer pair rather than requiring manual execution, satisfying the SI-7(1) frequency requirement discussed in Section 2.2 (Tables 3 and 4).

```
# /etc/systemd/system/fim-check.service
[Unit]
```

```

Description=File Integrity Monitor - integrity check
Wants=fim-check.timer

[Service]
Type=oneshot
ExecStart=/usr/bin/python3 /opt/fim/fim.py check -c /opt/fim/fim_config.json
User=root

[Install]
WantedBy=multi-user.target

```

Table 3: *fim-check.service systemd unit*

```

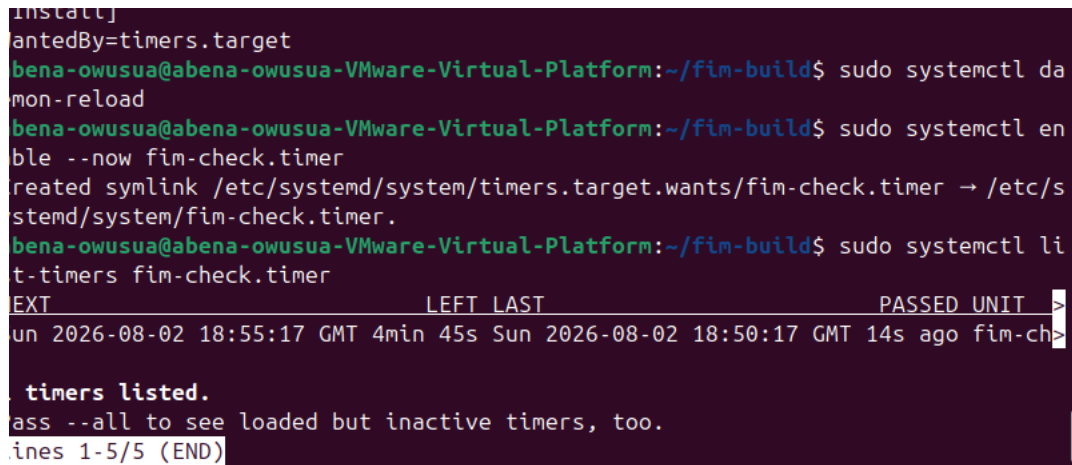
# /etc/systemd/system/fim-check.timer
[Unit]
Description=Run File Integrity Monitor check every 5 minutes

[Timer]
OnBootSec=2min
OnUnitActiveSec=5min
Persistent=true

[Install]
WantedBy=timers.target

```

Table 4: *fim-check.timer systemd unit*



```

[Install]
WantedBy=timers.target
bena-owusua@bena-owusua-VMware-Virtual-Platform:~/fim-build$ sudo systemctl da
mon-reload
bena-owusua@bena-owusua-VMware-Virtual-Platform:~/fim-build$ sudo systemctl en
ble --now fim-check.timer
Created symlink /etc/systemd/system/timers.target.wants/fim-check.timer → /etc/s
ystemd/system/fim-check.timer.
bena-owusua@bena-owusua-VMware-Virtual-Platform:~/fim-build$ sudo systemctl li
st-timers fim-check.timer
EXT                LEFT LAST                PASSED UNIT
-----
Sun 2026-08-02 18:55:17 GMT 4min 45s Sun 2026-08-02 18:50:17 GMT 14s ago fim-ch>

timers listed.
Pass --all to see loaded but inactive timers, too.
Lines 1-5/5 (END)

```

Figure 11: *systemctl enabling fim-check.timer and list-timers confirming NEXT and LAST run*

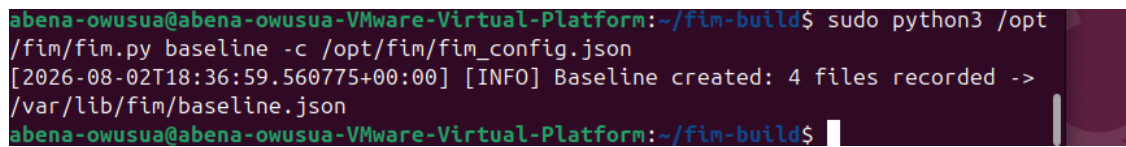
5. Results and Findings

This section presents the core evidence that the system correctly builds and protects a baseline, detects unauthorized changes, resists tampering, and operates both on a schedule and in real time. Each test was run in the isolated virtual machine described in Section 3.1, using the configuration in Table 2.

5.1 Baseline creation

The initial baseline was generated immediately after environment setup, while the system was in a known-clean state. This is the most security-critical step in the whole workflow, since baselining an already-compromised system would treat any existing backdoor as normal from that point onward.

```
$ sudo python3 /opt/fim/fim.py baseline -c /opt/fim/fim_config.json
```



```
abena-owusua@abena-owusua-VMware-Virtual-Platform:~/fim-build$ sudo python3 /opt/fim/fim.py baseline -c /opt/fim/fim_config.json
[2026-08-02T18:36:59.560775+00:00] [INFO] Baseline created: 4 files recorded -> /var/lib/fim/baseline.json
abena-owusua@abena-owusua-VMware-Virtual-Platform:~/fim-build$
```

Figure 5: Baseline creation output: "Baseline created: 4 files recorded -> /var/lib/fim/baseline.json"

Figure 6: Baseline files on disk

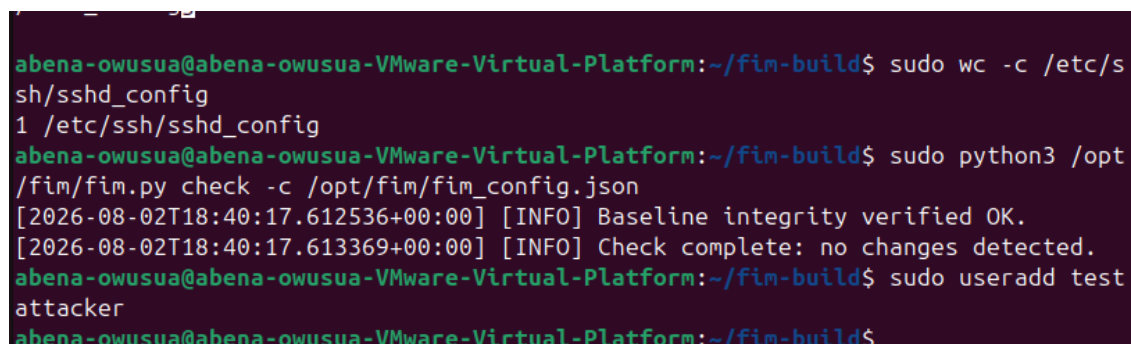
Table 5 shows an example baseline entry, for /etc/shadow, illustrating the metadata and hash captured per file.

```
"/etc/shadow": {
  "path": "/etc/shadow",
  "size": 1345,
  "mode": "0o640",
  "uid": 0,
  "gid": 42,
  "mtime": 1785687908.242207,
  "is_symlink": false,
  "symlink_target": null,
  "hash": "e173117df404e0c5b92a965ce4713f096b69c4ca502c8a380f65db0d20f340d5"
}
```

Table 5: Example baseline entry captured for /etc/shadow

5.2 Clean-state baseline check

With no changes made, a check against the fresh baseline correctly reported no findings, and the baseline's own self-hash verified successfully before the comparison ran, confirming the tamper-evidence step from Section 4.2 executes on every check, not only when something has changed.



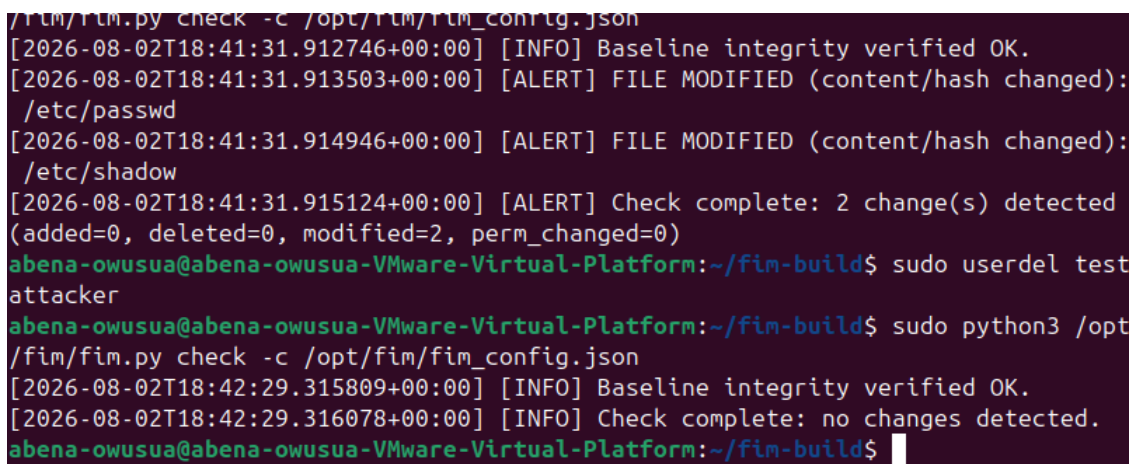
```
abena-owusua@abena-owusua-VMware-Virtual-Platform:~/fim-build$ sudo wc -c /etc/s
sh/sshd_config
1 /etc/ssh/sshd_config
abena-owusua@abena-owusua-VMware-Virtual-Platform:~/fim-build$ sudo python3 /opt/fim/fim.py check -c /opt/fim/fim_config.json
[2026-08-02T18:40:17.612536+00:00] [INFO] Baseline integrity verified OK.
[2026-08-02T18:40:17.613369+00:00] [INFO] Check complete: no changes detected.
abena-owusua@abena-owusua-VMware-Virtual-Platform:~/fim-build$ sudo useradd test
attacker
abena-owusua@abena-owusua-VMware-Virtual-Platform:~/fim-build$
```

Figure 7: Clean-state check: baseline integrity verified OK, no changes detected

5.3 Simulated attack: unauthorized account creation

To simulate a realistic attacker action, creating a backdoor account, a new user was added via `useradd`, which modifies both `/etc/passwd` and `/etc/shadow`. The next check correctly flagged both files as modified and reported the change count, matching the T1565.001 stored-data-manipulation pattern discussed in Section 2.1 (Figure 8).

```
$ sudo useradd testattacker
$ sudo python3 /opt/fim/fim.py check -c /opt/fim/fim_config.json
[ALERT] FILE MODIFIED (content/hash changed): /etc/passwd
[ALERT] FILE MODIFIED (content/hash changed): /etc/shadow
[ALERT] Check complete: 2 change(s) detected (added=0, deleted=0, modified=2,
perm_changed=0)
```

A terminal window showing the execution of the FIM tool. The first part shows the creation of a user 'testattacker' and a subsequent check that flags modifications to /etc/passwd and /etc/shadow. The second part shows the removal of the user with 'userdel' and a subsequent check that reports no changes, indicating a successful revert to the baseline.

```
/opt/fim/fim.py check -c /opt/fim/fim_config.json
[2026-08-02T18:41:31.912746+00:00] [INFO] Baseline integrity verified OK.
[2026-08-02T18:41:31.913503+00:00] [ALERT] FILE MODIFIED (content/hash changed):
/etc/passwd
[2026-08-02T18:41:31.914946+00:00] [ALERT] FILE MODIFIED (content/hash changed):
/etc/shadow
[2026-08-02T18:41:31.915124+00:00] [ALERT] Check complete: 2 change(s) detected
(added=0, deleted=0, modified=2, perm_changed=0)
abena-owusua@abena-owusua-VMware-Virtual-Platform:~/fim-build$ sudo userdel test
attacker
abena-owusua@abena-owusua-VMware-Virtual-Platform:~/fim-build$ sudo python3 /opt
/fim/fim.py check -c /opt/fim/fim_config.json
[2026-08-02T18:42:29.315809+00:00] [INFO] Baseline integrity verified OK.
[2026-08-02T18:42:29.316078+00:00] [INFO] Check complete: no changes detected.
abena-owusua@abena-owusua-VMware-Virtual-Platform:~/fim-build$
```

Figure 8: Attack simulation: both `/etc/passwd` and `/etc/shadow` flagged as `MODIFIED` after `useradd`, followed by the post-revert clean check

This confirms the tool correctly detects an unauthorized account being created, a common early step in a real intrusion.

5.4 Finding: reverted changes are not retroactively visible

After removing the test account with `userdel`, a subsequent check reported no changes, since `userdel` restored `/etc/passwd` and `/etc/shadow` to content identical to the original baseline (visible in the lower half of Figure 8). This is an expected property of state-based, snapshot-comparison FIM, and an important limitation to document: if an attacker reverts their changes before the next check runs, no evidence of the intermediate tampering remains in a single check result. This underlines the value of frequent scheduled checks and, for high-value targets, real-time monitoring, discussed further in Section 5.6 and Section 6.

5.5 Baseline tamper-resistance test

To validate that the baseline itself cannot be silently altered, its file was directly modified outside of the tool, by appending text to it, and a check was then run against it:

```
$ sudo bash -c 'echo "tampered" >> /var/lib/fim/baseline.json'
$ sudo python3 /opt/fim/fim.py check -c /opt/fim/fim_config.json
[ALERT] BASELINE TAMPERING DETECTED: baseline file hash mismatch!
```

Figure 9: Baseline tamper-resistance test

The check halts immediately rather than comparing against a baseline that may have been rewritten to hide an intrusion, exactly as designed in Section 4.2. The baseline was then restored via a fresh baseline run, visible as the "Baseline created" entry immediately following the tampering alert in Appendix A.

5.6 Real-time detection (watch mode)

Using the inotify-based watch mode, a file change to `/etc/hosts` was triggered from a second terminal while the watch process ran continuously in the first, so that detection could be observed without waiting for the next scheduled check:

```
Terminal 1:
$ sudo python3 /opt/fim/fim.py watch -c /opt/fim/fim_config.json
[INFO] Real-time watch started on 4 path(s). Press Ctrl+C to stop.
[ALERT] FILE MODIFIED (content/hash changed): /etc/hosts

Terminal 2 (triggering the change):
$ sudo bash -c 'echo "127.0.0.1 testentry" >> /etc/hosts'
```

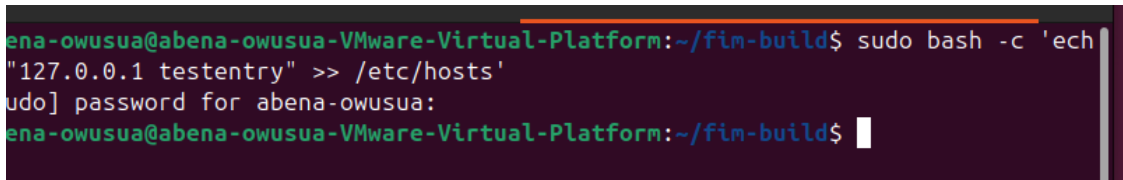
A screenshot of a terminal window with a dark background and light-colored text. The prompt is 'abena-owusua@abena-owusua-VMware-Virtual-Platform:~/fim-build\$'. The command entered is 'sudo bash -c \'echo "127.0.0.1 testentry" >> /etc/hosts\''. The output shows the password prompt 'sudo] password for abena-owusua:' followed by a new prompt 'abena-owusua@abena-owusua-VMware-Virtual-Platform:~/fim-build\$'.

Figure 10: Terminal 2: the command used to trigger a real-time file change on `/etc/hosts`

This demonstrates the system can provide sub-second detection for its most sensitive monitored paths, addressing the polling-delay limitation noted in Section 5.4.

5.7 Automated, unattended checks

The systemd timer configured in Section 4.6 was confirmed to fire on schedule without any manual command being run, as shown in Figure 11 (NEXT and LAST run times) and corroborated by the corresponding INFO entries in the full session log (Appendix A, Figure 12), which show a check completing at the same timestamp the timer reports as its LAST run.

5.8 Summary of test results

Table 7 summarizes the expected outcome and observed result for every scenario tested in this section, providing a single point of reference before the results are interpreted in Section 6.

Test scenario	Expected outcome	Result
---	---	---
5.1 Baseline creation	4 files recorded, self-hash stored	PASS
5.2 Clean-state check	No changes reported	PASS
5.3 Attack (useradd)	/etc/passwd and /etc/shadow flagged	PASS
5.4 Revert (userdel) (expected)	No changes reported (documented limit)	PASS
5.5 Baseline tampering	Tampering detected, check halted	PASS
5.6 Real-time watch mode	Change on /etc/hosts flagged instantly	PASS
5.7 Automated timer run	Check fires unattended on schedule	PASS

Table 7: Summary of test scenarios and results

6. Analysis and Recommendations

Taken together, the five test scenarios in Section 5 show that the system meets the objective set out in Section 1: it detects unauthorized changes to a defined set of critical files, resists having its own baseline silently rewritten, and can operate either on a schedule or in real time. Three findings, however, matter more than a simple pass/fail reading of the test results and shape the recommendations below.

6.1 What the results mean

First, the account-creation test (Section 5.3) confirms the tool detects the specific ATT&CK technique it was designed around, T1565.001 Stored Data Manipulation, with no false negatives across the two files useradd touches. That is a meaningful result on its own, but Section 5.4's finding, that a reverted change leaves no trace in a single check, is arguably the more operationally important one: it means the value of this control depends heavily on how frequently it runs, not just on whether it runs at all. A five-minute polling interval, as configured in Section 4.6, would still miss an attacker who creates and removes a backdoor account within that window, which is well within reach of a scripted attack.

Second, the tamper-resistance test (Section 5.5) validates the single design decision this report has emphasized most, that the baseline's own integrity is checked before it is trusted. Without that check, an attacker who successfully modifies a monitored file could, in principle, also patch the baseline to match, and every subsequent check would report a false all-clear. The test shows the tool refuses to do that silently. It does not, however, address a more fundamental version of the same problem: the baseline lives on the same host it protects (Section 3.2), so a sufficiently privileged local attacker could delete baseline.json and baseline.json.sha256 together rather than editing them, which the current design cannot distinguish from a legitimate reset. This is the most important open gap and is addressed in Recommendation 6.2.1 below.

Third, the real-time test (Section 5.6) confirms that watch mode closes the polling-delay gap identified above, but only for the paths it actively watches. In this deployment, that is the same four paths covered by scheduled polling, so the practical trade-off going into production would be deciding which subset of a much larger monitored-file list, for example just /etc/passwd,

/etc/shadow, and /etc/sudoers, justifies the continuous CPU and inotify-watch overhead of real-time coverage, versus which subset can safely rely on the five-minute scheduled scan.

6.2 Recommendations

6.2.1 Store the baseline off-host. The single highest-value change would be pushing the baseline and its hash to a separate host or to read-only media immediately after each baseline run, so that deleting or editing both files together on the monitored host is no longer sufficient to defeat the control. This directly closes the gap identified in Section 6.1.

6.2.2 Replace the self-hash with a signature. A SHA-256 self-hash proves the baseline has not changed since it was last hashed, but anyone with write access to both baseline.json and baseline.json.sha256 can update both consistently. A GPG or RSA signature, verified with a private key that never touches the monitored host, would raise the bar from "the baseline is internally consistent" to "the baseline was produced by someone holding the signing key," a materially stronger guarantee.

6.2.3 Shorten the effective detection window for high-value paths, or extend real-time coverage. Since Section 5.4 showed that reverted changes between checks are invisible to polling, either the scheduled interval should be shortened for the highest-risk paths, or watch mode should be extended to cover a broader set of files than the four tested here, accepting the additional inotify and CPU overhead in exchange for closing the detection gap.

6.2.4 Forward alerts to a central SIEM. Alerting currently relies on the local log file, syslog, and optional webhook or email, all reachable from the monitored host itself. Forwarding syslog output to a centralized platform such as Wazuh, ELK, or Splunk would allow correlation across multiple hosts and would keep an alert record outside the reach of an attacker who compromises the monitored host after the fact.

6.2.5 Add a maintenance-window mechanism. Every test in Section 5 was run against a system with no legitimate background change activity. In production, routine patching, log rotation, and configuration management would all trigger the same ALERT-level output as a genuine attack, and without a way to suppress expected changes during a declared maintenance window, the volume of false-positive alerts would quickly train operators to ignore them, a well-documented failure mode for any detection control.

6.2.6 Benchmark against an established tool. Running AIDE or Tripwire in parallel against the same monitored paths and comparing detection coverage and timing would validate that the custom implementation in this project has no blind spots that an established, widely audited tool would have caught.

7. Conclusion

This project set out to design, build, and test a working File Integrity Monitoring system from first principles, rather than deploying an existing tool unmodified, and to document the reasoning behind the detection logic, tamper-resistance requirements, and operational

deployment considerations involved. That objective was met: fim.py successfully baselines a defined set of critical Linux configuration files, protects that baseline with a self-hash checked on every run, classifies changes by type, and supports both scheduled and real-time detection.

Across five test scenarios, covering a clean-state check, a simulated backdoor-account attack, a baseline tamper-resistance test, unattended scheduled operation, and real-time event-driven detection, the system produced the expected result in every case. The one limitation surfaced by testing, that state-based comparison cannot retroactively reveal changes reverted between checks, is a known property of this class of tool rather than an implementation defect, and Section 6 sets out concrete steps, chiefly moving the baseline off-host and tightening the effective detection window, that would close the remaining gaps if this system were taken from a course project into a production deployment.

More broadly, the exercise reinforced why FIM is treated as a baseline control by frameworks such as CIS and NIST SP 800-53 rather than an optional extra: it is one of the few controls that can catch an intrusion by what an attacker had to change on disk, independent of whether the initial access vector itself was ever detected.

References

- [1] MITRE ATT&CK. "Data Manipulation, Technique T1565 - Enterprise." MITRE Corporation. Available: <https://attack.mitre.org/techniques/T1565/>
- [2] MITRE ATT&CK. "Create or Modify System Process, Technique T1543 - Enterprise." MITRE Corporation. Available: <https://attack.mitre.org/techniques/T1543/>
- [3] MITRE ATT&CK. "Masquerading, Technique T1036 - Enterprise." MITRE Corporation. Available: <https://attack.mitre.org/techniques/T1036/>
- [4] Center for Internet Security. "CIS Distribution Independent Linux Benchmark," recommendation 1.3.2, "Ensure filesystem integrity is regularly checked." Available: <https://www.cisecurity.org/cis-benchmarks>
- [5] National Institute of Standards and Technology. "Security and Privacy Controls for Information Systems and Organizations," NIST Special Publication 800-53, Rev. 5, control SI-7 and SI-7(1), Software, Firmware, and Information Integrity.
- [6] AIDE Project. "Advanced Intrusion Detection Environment." Available: <https://aide.github.io/>
- [7] Wazuh Inc. "File Integrity Monitoring." Wazuh Documentation. Available: <https://documentation.wazuh.com/>
- [8] Python watchdog contributors. "watchdog: Python API and shell utilities to monitor file system events." PyPI. Available: <https://pypi.org/project/watchdog/>

Appendix A: Full Test Timeline (Log Evidence)

The complete fim.log captured across the testing session provides an end-to-end audit trail covering every scenario in Section 5, from the first clean baseline through the attack simulation, the reverted-change check, the tamper-resistance test, baseline recreation, the automated timer run, and the real-time watch-mode alert.

```
[18:40:17] [INFO] Baseline integrity verified OK.
[18:40:17] [INFO] Check complete: no changes detected.
[18:41:31] [ALERT] FILE MODIFIED: /etc/passwd
[18:41:31] [ALERT] FILE MODIFIED: /etc/shadow
[18:41:31] [ALERT] Check complete: 2 change(s) detected
[18:42:29] [INFO] Check complete: no changes detected. (post-revert)
[18:44:19] [ALERT] BASELINE TAMPERING DETECTED
[18:44:56] [INFO] Baseline created: 4 files recorded
[18:45:11] [INFO] Check complete: no changes detected.
[18:50:17] [INFO] Check complete: no changes detected. (automated timer run)
[18:56:40] [ALERT] FILE MODIFIED: /etc/hosts (real-time watch mode)
```

Table 6: Full fim.log test-session timeline (summarised)

```
2026-08-02T18:40:17.612536+00:00] [INFO] Baseline integrity verified OK.
2026-08-02T18:40:17.613369+00:00] [INFO] Check complete: no changes detected.
2026-08-02T18:41:31.912746+00:00] [INFO] Baseline integrity verified OK.
2026-08-02T18:41:31.913503+00:00] [ALERT] FILE MODIFIED (content/hash changed):
/etc/passwd
2026-08-02T18:41:31.914946+00:00] [ALERT] FILE MODIFIED (content/hash changed):
/etc/shadow
2026-08-02T18:41:31.915124+00:00] [ALERT] Check complete: 2 change(s) detected
added=0, deleted=0, modified=2, perm_changed=0)
2026-08-02T18:42:29.315809+00:00] [INFO] Baseline integrity verified OK.
2026-08-02T18:42:29.316078+00:00] [INFO] Check complete: no changes detected.
2026-08-02T18:44:19.388193+00:00] [ALERT] BASELINE TAMPERING DETECTED: baseline
file hash mismatch!
2026-08-02T18:44:56.592504+00:00] [INFO] Baseline created: 4 files recorded ->
var/lib/fim/baseline.json
2026-08-02T18:45:11.711488+00:00] [INFO] Baseline integrity verified OK.
2026-08-02T18:45:11.712091+00:00] [INFO] Check complete: no changes detected.
2026-08-02T18:50:17.269045+00:00] [INFO] Baseline integrity verified OK.
2026-08-02T18:50:17.269468+00:00] [INFO] Check complete: no changes detected.
bena-owusua@bena-owusua-VMware-Virtual-Platform:~/fim-build$
```

Figure 12: fim.log excerpt captured directly from the test session

Appendix B: Full Source Code Reference

The complete `fim.py` implementation is included as a separate attachment alongside this report and in the project's GitHub repository (see cover page for the repository URL), under `src/fim.py`.